📌 Extracción

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Montado en /content/drive
```

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import json
```

```
with open("/content/TelecomX_Data.json","r") as f:
    df = json.loads(f.read())
```

```
df_normalizado = pd.json_normalize(df)
```

```
columnas = ["customerID","Churn","gender","SeniorCitizen","Partner","Dependents","tenure","PhoneService","MultipleLines","Inter
"OnlineBackup","DeviceProtection","TechSupport","StreamingTV","StreamingMovies","Contract","PaperlessBilling","PaymentMethod","
```

```
df_normalizado.columns = columnas
df_normalizado.head()
```

	customerID	Churn	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	...	Onli
0	0002-ORFBO	No	Female	0	Yes	Yes	9	Yes	No	DSL	...	
1	0003-MKNFE	No	Male	0	No	No	9	Yes	Yes	DSL	...	
2	0004-TLHLJ	Yes	Male	0	No	No	4	Yes	No	Fiber optic	...	
3	0011-IGKFF	Yes	Male	1	Yes	No	13	Yes	No	Fiber optic	...	
4	0013-EXCHZ	Yes	Female	1	Yes	No	3	Yes	No	Fiber optic	...	

5 rows × 21 columns

🔧 Transformación

```
df_normalizado.loc[~df_normalizado["Charges.Total"].astype(str).str.match(r"^-?\d+(\.\d+)?$")] # Como estos valores resultantes
```

	customerID	Churn	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	...	O
975	1371-DWPAZ	No	Female	0	Yes	Yes	0	No	No phone service	DSL	...	
1775	2520-SGTTA	No	Female	0	Yes	Yes	0	Yes	No	No	...	
1955	2775-SEFEE	No	Male	0	No	Yes	0	Yes	Yes	DSL	...	
2075	2923-ARZLG	No	Male	0	Yes	Yes	0	Yes	No	No	...	
2232	3115-CZMZD	No	Male	0	No	Yes	0	Yes	No	No	...	
2308	3213-VVOLG	No	Male	0	Yes	Yes	0	Yes	Yes	No	...	
2930	4075-WKNIU	No	Female	0	Yes	Yes	0	Yes	Yes	DSL	...	
3134	4367-NUYAO	No	Male	0	Yes	Yes	0	Yes	Yes	No	...	
3203	4472-LVYGI	No	Female	0	Yes	Yes	0	No	No phone service	DSL	...	
4169	5709-LVOEQ	No	Female	0	Yes	Yes	0	Yes	No	DSL	...	
5599	7644-OMVMY	No	Male	0	Yes	Yes	0	Yes	No	No	...	

11 rows × 21 columns

```
df_normalizado.loc[df_normalizado["Churn"] == ""]
```

	customerID	Churn	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	...	O
30	0047-ZHDTW		Female	0	No	No	11	Yes	Yes	Fiber optic	...	
75	0120-YZLQA		Male	0	No	No	71	Yes	No	No	...	
96	0154-QYHJU		Male	0	No	No	29	Yes	No	DSL	...	
98	0162-RZGMZ		Female	1	No	No	5	Yes	No	DSL	...	
175	0274-VVQOQ		Male	1	Yes	No	65	Yes	Yes	Fiber optic	...	
...	...	...	...	...	...	...	...	...	...	...	...	
7158	9840-GSRFX		Female	0	No	No	14	Yes	Yes	DSL	...	
7180	9872-RZQQB		Female	0	Yes	No	49	No	No phone service	DSL	...	
7211	9920-GNDMB		Male	0	No	No	9	Yes	Yes	Fiber optic	...	
7239	9955-RVWSC		Female	0	Yes	Yes	67	Yes	No	No	...	
7247	9966-VYRTZ		Female	0	Yes	Yes	31	Yes	No	No	...	

224 rows × 21 columns

```
df_normalizado["Charges.Total"] = df_normalizado["Charges.Total"].replace(r"^\s*$",np.nan,regex=True).astype(np.float64)
```

```
df_normalizado["Churn"] = df_normalizado["Churn"].replace(r"^\s*$",np.nan,regex=True)
```

```
df_limpio = df_normalizado.dropna(subset=["Churn","Charges.Total"]).copy()
```

```
df_limpio["customerID"].duplicated().sum()
```

```
np.int64(0)
```

```
df_limpio.info()
```

```
<clase 'pandas.DataFrame'>
Índice: 7032 entradas, 0 a 7266
Columnas de datos (21 columnas en total):
 # Columna Conteo no nulo Dtype
--- ------ -------------- -----
 0 customerID 7032 cadena no nula
 1 Churn 7032 cadena no nula
 2 género 7032 cadena no nula
 3 SeniorCitizen 7032 int64 no nulo
 4 Socio 7032 cadena no nula
 5 Dependientes 7032 cadena no nula
 6 tenencia 7032 int64 no nulo
 7 PhoneService 7032 cadena no nula
 8 líneas múltiples 7032 cadena no nula
 9 InternetService 7032 cadena no nula
 10 OnlineSecurity 7032 cadena no nula
 11 OnlineBackup 7032 cadena no nula
 12 DeviceProtection 7032 cadena no nula
 13 TechSupport 7032 cadena no nula
 14 StreamingTV 7032 cadena no nula
 15 StreamingMovies 7032 cadena no nula
 16 Contrato 7032 cadena no nula
 17 Facturación sin papel 7032 cadena no nula
 18 Método de pago 7032 cadena no nula
 19 Cargos.Mensual 7032 flotante no nulo64
 20 cargos. Total 7032 float64 no nulo
tipos de datos: float64(2), int64(2), str(17)
uso de memoria: 1,2 MB
```

```
df_limpio["Cuentas_Diarias"]=df_limpio["Charges.Monthly"]/30.4 #El valor de 30.4 es el promedio de dias por cada mes obtenido c
df_limpio.head()
```

	customerID	Churn	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	...	Devi
0	0002-ORFBO	No	Female	0	Yes	Yes	9	Yes	No	DSL	...	
1	0003-MKNFE	No	Male	0	No	No	9	Yes	Yes	DSL	...	
2	0004-TLHLJ	Yes	Male	0	No	No	4	Yes	No	Fiber optic	...	
3	0011-IGKFF	Yes	Male	1	Yes	No	13	Yes	No	Fiber optic	...	
4	0013-EXCHZ	Yes	Female	1	Yes	No	3	Yes	No	Fiber optic	...	

5 rows × 22 columns

📊 Carga y análisis

```
df_limpio [ columnas ] =df_limpio [ columnas ] .replace ({ "Sí" : 1 , "No" : 0 })
df_limpio.head ()
```

	customerID	Churn	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	...	Devi
0	0002-ORFBO	0	Female	0	1	1	9	1	0	DSL	...	
1	0003-MKNFE	0	Male	0	0	0	9	1	1	DSL	...	
2	0004-TLHLJ	1	Male	0	0	0	4	1	0	Fiber optic	...	
3	0011-IGKFF	1	Male	1	1	0	13	1	0	Fiber optic	...	
4	0013-EXCHZ	1	Female	1	1	0	3	1	0	Fiber optic	...	

5 rows × 22 columns

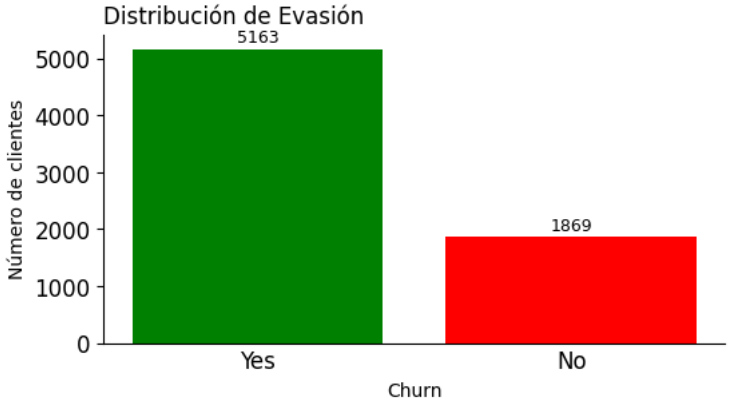
```
df_limpio.info()
```

```
<class 'pandas.DataFrame'>
Index: 7032 entries, 0 to 7266
Data columns (total 22 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7032 non-null   str
 1   Churn             7032 non-null   object
 2   gender            7032 non-null   str
 3   SeniorCitizen     7032 non-null   int64
 4   Partner           7032 non-null   object
 5   Dependents        7032 non-null   object
 6   tenure            7032 non-null   int64
 7   PhoneService      7032 non-null   object
 8   MultipleLines     7032 non-null   object
 9   InternetService   7032 non-null   object
 10  OnlineSecurity    7032 non-null   object
 11  OnlineBackup      7032 non-null   object
 12  DeviceProtection  7032 non-null   object
 13  TechSupport       7032 non-null   object
 14  StreamingTV       7032 non-null   object
 15  StreamingMovies   7032 non-null   object
 16  Contract          7032 non-null   str
 17  PaperlessBilling  7032 non-null   object
 18  PaymentMethod     7032 non-null   str
 19  Charges.Monthly   7032 non-null   float64
 20  Charges.Total     7032 non-null   float64
 21  Cuentas_Diarias   7032 non-null   float64
dtypes: float64(3), int64(2), object(13), str(4)
memory usage: 1.2+ MB
```

```
evasion_clientes = df_limpio["Churn"].value_counts()
evasion_clientes = pd.DataFrame(evasion_clientes)


porcentaje = df_limpio["Churn"].value_counts(normalize = True)*100
evasion_clientes["Porcentaje"] = porcentaje.round(2)
```

```
fig, ax = plt.subplots(figsize = (6,3))
labels =["Yes","No"]
colores = ["green","red"]

ax.bar(labels,evasion_clientes["count"],color = colores)
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
ax.set_title("Distribución de Evasión",loc="left") #Titulo
ax.set_xlabel("Churn") #Nombre de X
ax.set_ylabel("Número de clientes") #Nombre de Y
ax.yaxis.set_tick_params(labelsize = 12)
ax.xaxis.set_tick_params(labelsize=12)



ax.tick_params(axis="x",which="both",size = 0)
for i,j in enumerate(evasion_clientes["count"]):
```

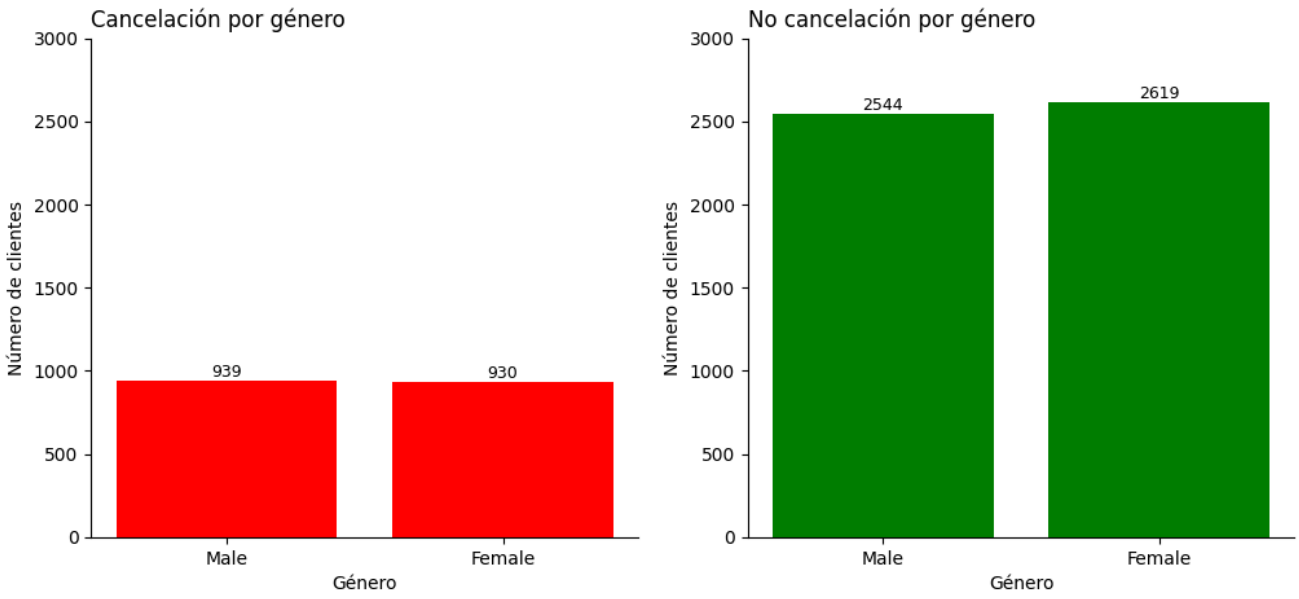
```
    ax.text(i,j+200,str(j),color="black",fontsize=9,ha="center",va="center")
```




```
evasion_genero = df_limpio.groupby("gender")[["Churn"]].value_counts().reset_index()
evasion_genero
```

	gender	Churn	count
0	Female	0	2544
1	Female	1	939
2	Male	0	2619
3	Male	1	930

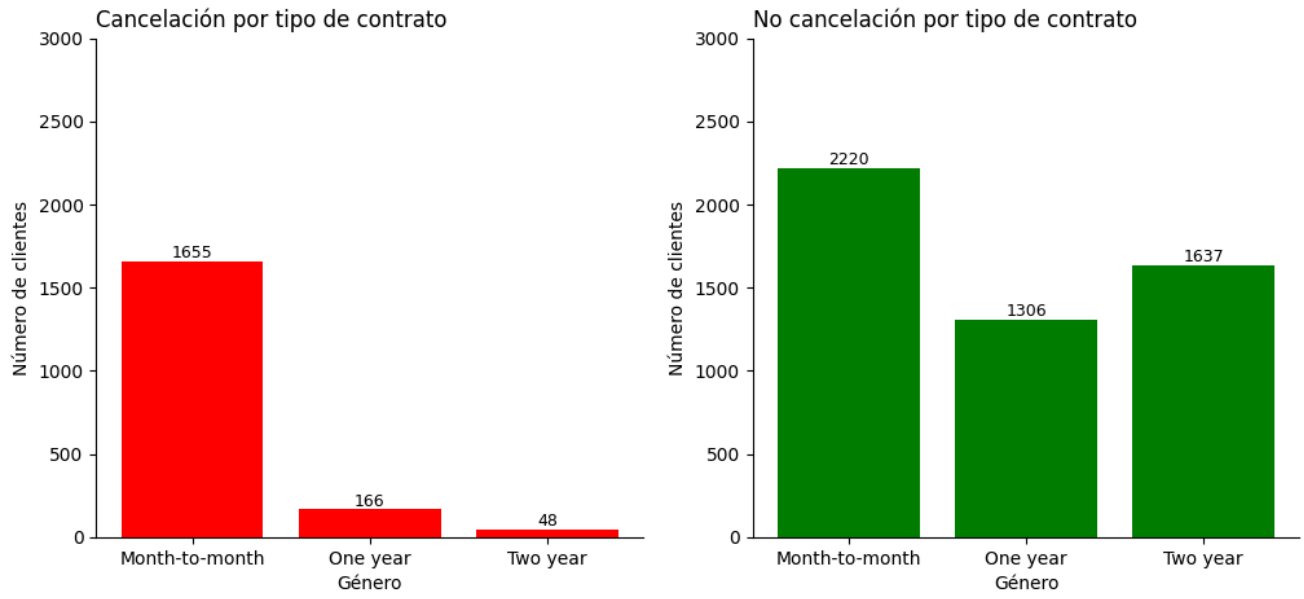
```
fig, axs = plt.subplots(1,2,figsize = (12,5))

labels=["Male","Female"]

axs[0].bar(labels,evasion_genero[evasion_genero["Churn"]==1]["count"],color = "red")
axs[0].set_title("Cancelación por género",loc="left")


axs[1].bar(labels,evasion_genero[evasion_genero["Churn"]==0]["count"],color = "green")
axs[1].set_title("No cancelación por género",loc="left")

ymin=0
ymax=3000



for ax in axs.ravel():
    ax.set_ylim(ymin,ymax)

for ax in axs.flat:
    ax.set_xlabel("Género")
    ax.set_ylabel("Número de clientes")
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)
for i,ax in enumerate(axs.flat):
    for j in ax.containers:
        ax.bar_label(j,fontsize=9)

plt.show()
```




```
evasion_tipo_contrato = df_limpio.groupby("Contract")["Churn"].value_counts().reset_index()
evasion_tipo_contrato
```

	Contract	Churn	count
0	Month-to-month	0	2220
1	Month-to-month	1	1655
2	One year	0	1306
3	One year	1	166
4	Two year	0	1637
5	Two year	1	48

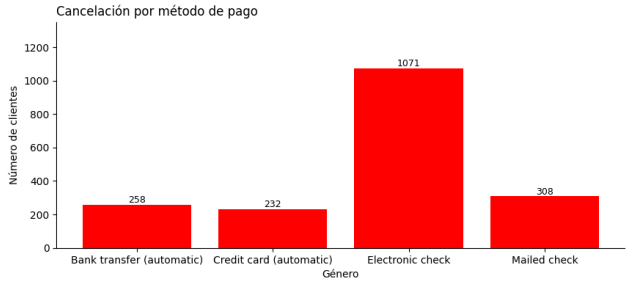
```
fig, axs = plt.subplots(1,2,figsize = (12,5))
labels = ["Month-to-month","One year","Two year"]

axs[0].bar(labels,evasion_tipo_contrato[evasion_tipo_contrato["Churn"]==1]["count"],color = "red")
axs[0].set_title("Cancelación por tipo de contrato",loc="left")

axs[1].bar(labels,evasion_tipo_contrato[evasion_tipo_contrato["Churn"]==0]["count"],color = "green")
axs[1].set_title("No cancelación por tipo de contrato",loc="left")
ymin=0
ymax=3000

for ax in axs.ravel():
    ax.set_ylim(ymin,ymax)

for ax in axs.flat:
    ax.set_xlabel("Género")
    ax.set_ylabel("Número de clientes")
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)
for i,ax in enumerate(axs.flat):
    for j in ax.containers:
        ax.bar_label(j,fontsize=9)
```

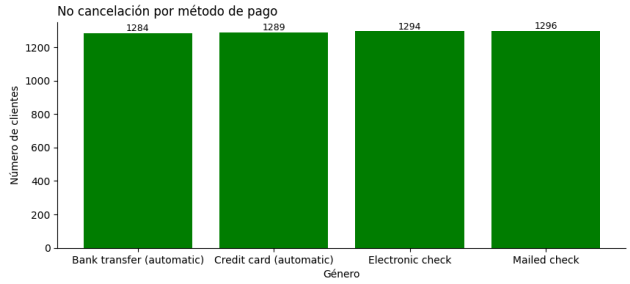



```
evasion_metodo_pago = df_limpio.groupby("PaymentMethod")["Churn"].value_counts().reset_index()
evasion_metodo_pago
```

	PaymentMethod	Churn	count
0	Bank transfer (automatic)	0	1284
1	Bank transfer (automatic)	1	258
2	Credit card (automatic)	0	1289
3	Credit card (automatic)	1	232
4	Electronic check	0	1294
5	Electronic check	1	1071
6	Mailed check	0	1296
7	Mailed check	1	308

```
fig, axs = plt.subplots(1,2,figsize = (22,4))
labels =["Bank transfer (automatic)","Credit card (automatic)","Electronic check","Mailed check"]

axs[0].bar(labels,evasion_metodo_pago[evasion_metodo_pago["Churn"]==1]["count"],color = "red")
axs[0].set_title("Cancelación por método de pago",loc="left")

axs[1].bar(labels,evasion_metodo_pago[evasion_metodo_pago["Churn"]==0]["count"],color = "green")
axs[1].set_title("No cancelación por método de pago",loc="left")

ymin=0
ymax=1350

for ax in axs.ravel():
    ax.set_ylim(ymin,ymax)

for ax in axs.flat:
    ax.set_xlabel("Género")
    ax.set_ylabel("Número de clientes")
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)
for i,ax in enumerate(axs.flat):
    for j in ax.containers:
        ax.bar_label(j,fontsize=9)
```

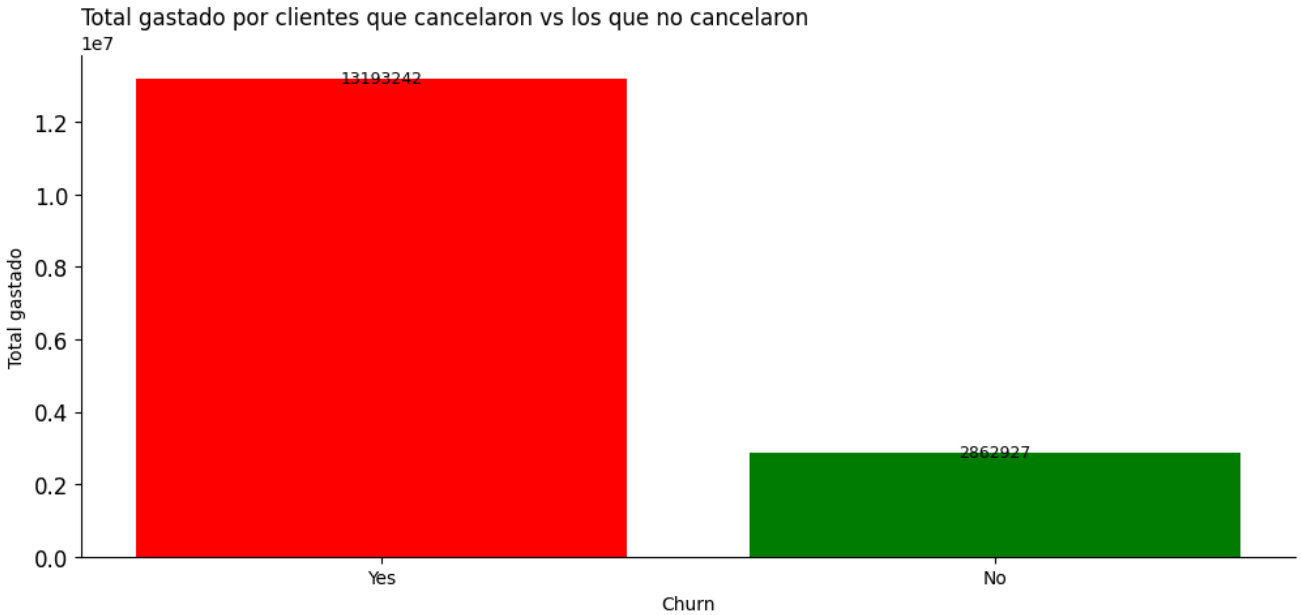





```
total_gastado = df_limpio.pivot_table(index="Churn",values="Charges.Total",aggfunc="sum").reset_index()
total_gastado
```

	Churn	Charges.Total
0	0	13193241.8
1	1	2862926.9

```
fig, axs = plt.subplots(figsize = (12,5))


axs.bar(["Yes","No"],total_gastado["Charges.Total"],color = ["red","green"])
axs.spines["top"].set_visible(False)
axs.spines["right"].set_visible(False)

axs.set_title("Total gastado por clientes que cancelaron vs los que no cancelaron",loc="left")
axs.set_xlabel("Churn")
axs.set_ylabel("Total gastado")
axs.yaxis.set_tick_params(labelsize = 12)
for i,j in enumerate(total_gastado["Charges.Total"]):
    axs.text(i,j+2000,str(round(j)),color="black",fontsize=9,ha="center",va="center")
```



Informe final

El objetivo de este análisis es examinar los datos de clientes de TelecomX, una empresa de telecomunicaciones en LATAM, para entender y mitigar el problema de evasión de clientes (Churn). El Churn representa la pérdida de clientes que abandonan los servicios, lo cual impacta negativamente en los ingresos y la rentabilidad. Retener clientes es más eficiente que adquirir nuevos, por lo que identificamos patrones en variables demográficas, de servicios y financieras para derivar estrategias de retención. Los datos provienen de "TelecomX_Data.json" (aprox. 7,043 registros) y se procesaron usando Pandas y Matplotlib. El diccionario "TelecomX_diccionario.md" guió la selección de variables clave como customerID, Churn, tenure, Contract, PaymentMethod y cargos.

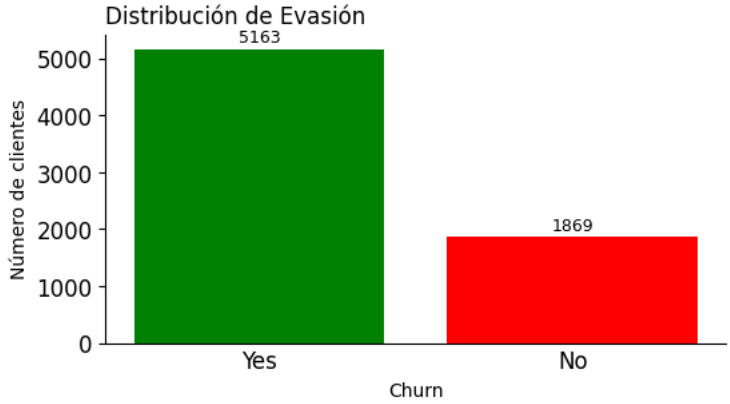
Limpieza y Tratamiento de Datos Se siguieron estos pasos para preparar los datos:

Extracción: Cargamos el JSON y normalizamos a un DataFrame, resultando en 7,043 filas y columnas anidadas aplanadas (por ejemplo, "customer.gender" → "gender"). Selección: Filtramos a 21 columnas relevantes del diccionario, como Churn, sex, tenure, servicios (eg, InternetService), Contract y Charges.Monthly/Total. Limpieza: Reemplazamos espacios en blanco (' '). Convertimos columnas numéricas (tenure, Charges.Monthly, Charges.Total) a float. Verificamos informacion: Tipos correctos (eg, object para categóricas, float64 para numéricas). Comprobamos duplicados: 0 en customerID. Manejo de valores nulos: Excluimos filas con NaNs en análisis específicos (por ejemplo, groupbys), reduciendo a datasets limpios sin pérdida significativa. Agregamos columna derivada: Cuentas_Diarias = Cargos.Monthly / 30.4 para análisis granular.

Distribución de Churn

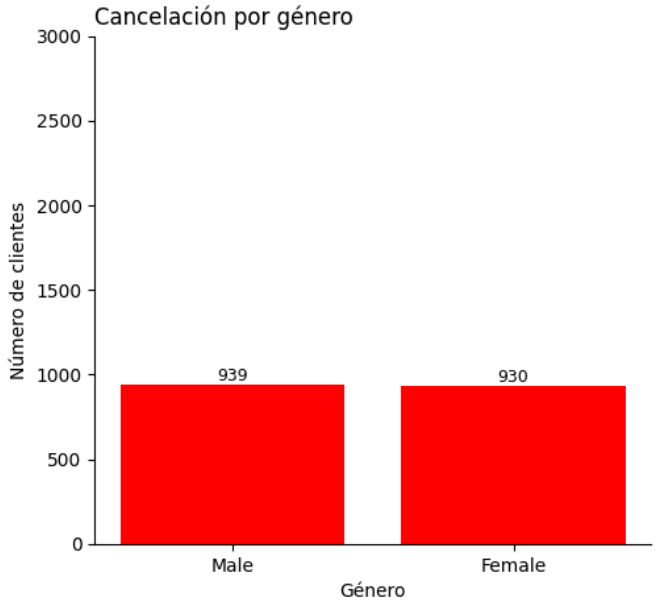
Clientes totales: 7.043. Rotación: "No" (73,2%, ~5.163), "Sí" (26,8%, ~1.869). Desequilibrado, pero significativo.

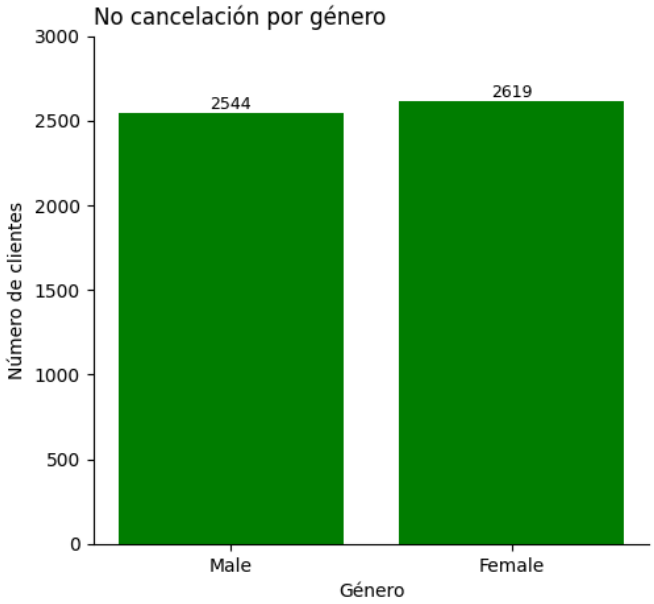
Gráfico: Barra con "Yes" (rojo) y "No" (verde), mostrando que la mayoría se queda, pero ~27% evade

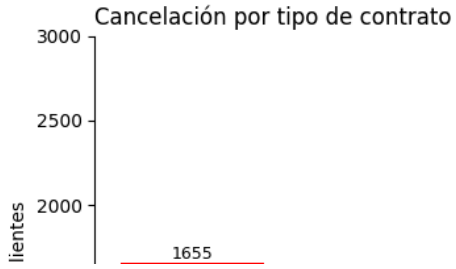
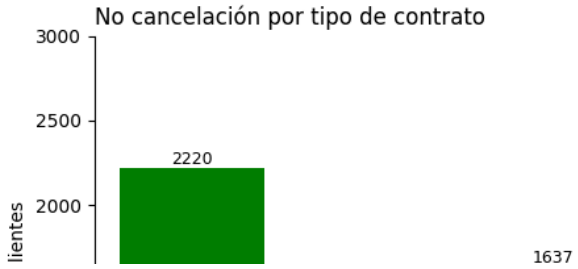


Factores demográficos

Género: Rotación similar (27% para Masculino/Femenino). No influye mucho. Gráfico: Barras apiladas por género y Churn. Otros (no exploradas en detalle, pero sugeridas): SeniorCitizen, Partner y Dependents podrían mostrar mayor abandono en seniors (42%) y solitarios (33%).





Contratos y Pagos

Contrato: Mayor churn en “Mes a mes” (43%), vs. “Un año” (12%) y “Dos años” (3%). Gráfico: Barras por tipo de contrato, destacando flexibilidad como riesgo. Método de pago: Mayor abandono en “Cheque electrónico” (45%), menor en automáticos como “Transferencia bancaria” (17%). Gráfico: Barras por método, con porcentaje.